

Exercice : algorithme de résolution d'un labyrinthe

Le but de cet exercice, qui devrait représenter un défi un peu plus important, est de développer un ou des algorithme(s) permettant de retrouver la sortie d'un labyrinthe généré aléatoirement. L'énoncé est volontairement plus générique que pour les autres exercices : il vous explique dans les grandes lignes comment la résolution doit fonctionner, mais en vous laissant la tâche de trouver l'algorithme exact.

L'exécutable « Exemple.exe » vous montre le résultat à atteindre. Le projet « JeuLabyrinthe » vous servira de projet de départ, dans lequel vous devez ajouter votre algorithme.

1 Interface

1.1 Interface graphique

Le projet été doté d'une interface graphique permettant de dessiner le labyrinthe, de choisir sa taille, ainsi que d'autres options. Il n'est pas nécessaire¹ pour l'exercice de comprendre la façon dont cette interface fonctionne, la manière dont le labyrinthe est implémenté, etc.

Ce que vous devez savoir :

- Tout le code qui vous concerne est dans le fichier « VotreSolutionIci.cs ». Vous pouvez ignorer tous les autres fichiers.
- Lorsque l'utilisateur clique le bouton « Résoudre », le programme appelle la fonction `Resoudre()`, qui doit calculer et renvoyer un trajet vers la sortie : à vous de programmer cette fonction. Rien ne vous empêche évidemment de découper la solution en d'autres fonctions appelées à partir de celle-ci. Une fois la fonction exécutée, le programme anime un déplacement selon le trajet qu'elle a renvoyé.
- Le bouton « Simplifier », qui appelle la fonction destinée à simplifier le trajet trouvé, fonctionne de la même manière.
- Si vous choisissez l'option « Labyrinthe prédéfini », le programme générera toujours le même labyrinthe, que vous pouvez changer en changeant le numéro situé juste à côté : ceci peut faciliter le *debugging* en permettant de se retrouver toujours dans les mêmes conditions. Choisir « Non » générera un labyrinthe aléatoire.
- Attention : votre fonction `Resoudre()` doit renvoyer tout le trajet d'un coup : tant que son exécution n'est pas terminée, l'interface graphique n'affiche rien.

¹ Cela dit, si vous êtes curieux, il n'est évidemment pas interdit d'aller voir 😊. Une bonne partie des concepts utilisés seront discutés dans le cours de niveau 2. Notez cependant que mes choix d'implémentation ne correspondent pas toujours aux procédures de programmation standard, principalement pour simplifier l'exercice.

1.2 Le labyrinthe

Remarque : sauf quand c'est explicitement précisé, toutes les directions sont indiquées de manière absolue : par exemple « gauche » correspond à la gauche de l'écran.

Vous allez devoir interagir avec le labyrinthe, pour vérifier où sont les murs, l'entrée, etc. Ceci va demander quelques éléments issus de la programmation orientée objet, mais rien de bien compliqué.

Chaque case du labyrinthe est implémentée par un objet de type Cell (cellule en anglais). Les seules fonctionnalités dont vous avez besoin sont celles permettant de localiser les murs. Ceci s'obtient en testant les 4 valeurs MurHaut, MurBas, MurGauche et MurDroite, qui sont des booléens indiquant s'il y a un mur dans la direction indiquée. Par exemple, si la variable CaseCourante est une case du labyrinthe et que CaseCourante.MurHaut est égal à true, cela signifie qu'il y a un mur en haut.

Le labyrinthe en lui-même est un tableau de cases (donc d'objets de type Cell). Vous aurez besoin :

- D'accéder aux diverses cases, via la fonction GetCell(), en lui indiquant les coordonnées horizontale et verticale de la case. Ceci fonctionne donc presque comme pour lire un tableau (mais GetCell() simplifie la gestion des cas particuliers).
- D'obtenir la position de l'entrée du labyrinthe, via PositionEntree() qui renvoie les coordonnées horizontale et verticale sous forme de paramètres out.
- D'obtenir la position de sortie du labyrinthe, via PositionSortie(), avec le même comportement.

Votre projet de départ contient quelques instructions pour vous aider à maîtriser cette interface. Remplacez ensuite cet exemple par votre solution.

1.3 La fonction Resoudre()

La fonction Resoudre() que vous devez programmer reçoit donc le labyrinthe à résoudre et doit renvoyer le trajet vers la sortie. Un trajet est implémenté comme une liste de char donnant la liste des mouvements à faire. Par exemple, si la liste vaut ('H', 'G', 'G'), cela signifie que le voyageur doit faire un pas vers le haut, puis deux pas vers la gauche.

2 Algorithmes de résolution de labyrinthe

2.1 Marche aléatoire (random walk)

C'est une solution très inefficace, mais qui a l'avantage d'être très simple à programmer : à chaque étape, on tire une direction au hasard. S'il n'y a pas de mur de ce côté, on fait un pas dans cette direction, puis on recommence. On continue jusqu'à la sortie.

Attention à ne pas sortir du labyrinthe par l'entrée !

2.2 Règle de la main droite

C'est une autre méthode assez intuitive : à l'entrée du labyrinthe, on pose la main droite sur le mur de droite et on avance sans lâcher le mur jusqu'à la sortie.

En d'autres termes, l'algorithme peut se décrire comme suit :

- Imaginer un voyageur se déplaçant dans le labyrinthe : il est tourné dans le sens dans lequel il vient d'avancer (à l'entrée, il est donc tourné vers le haut, car l'entrée du labyrinthe est toujours dans le mur inférieur).
- A chaque étape, il va considérer d'abord **sa** droite, puis devant lui, puis sa gauche, puis derrière lui. Il avance dans la première de ces directions où il n'y a pas de mur, puis recommence, jusqu'à atteindre la sortie.

Attention : les directions sont ici à considérer par rapport au voyageur. Sa droite ne correspond pas forcément à la droite de l'écran : par exemple, s'il vient de faire un pas vers la gauche, sa droite pointe vers le haut de l'écran.

3 Simplification du trajet

Une fois une solution trouvée, il est utile de la simplifier en éliminant les détours inutiles. Pour le type de labyrinthes générés ici, dans lesquels il n'y a qu'un seul chemin de l'entrée à la sortie, cela peut se faire assez simplement.

3.1 Interface de la fonction Simplifier()

La fonction Simplifier(), que vous devez également écrire, reçoit un trajet et doit renvoyer un autre trajet, éliminant les détours. Comme avant, ces trajets sont implémentés comme des listes de char.

Lorsque l'utilisateur clique le bouton « Simplifier », votre fonction est appelée, puis le programme anime un déplacement selon le trajet renvoyé.

3.2 Algorithme de simplification

L'algorithme de simplification consiste à éliminer tous les mouvements inutiles, comme le fait d'aller vers le haut, puis immédiatement après vers le bas.

L'algorithme peut être décrit très simplement :

- Nous allons parcourir le trajet original une seule fois, mais en faisant des marches arrière en cours de parcours.
- A chaque étape, si le mouvement décrit à la position courante est le contraire de celui de la position suivante, on supprime ces deux mouvements de la liste, puis on fait reculer la position courante de 1, car la suppression a peut-être entraîné une nouvelle situation simplifiable.
- L'algorithme est terminé lorsqu'on atteint la fin de la liste.

Exemple, avec le trajet ('H', 'G', 'H', 'B', 'D') :

1. *Position courante : 0, les mouvements considérés sont 'H' et 'G'. Ils ne sont pas le contraire l'un de l'autre, donc rien à faire.*
2. *Position courante : 1, les mouvements considérés sont 'G' et 'H'. Idem.*

3. *Position courante : 2, les mouvements considérés sont 'H' et 'B' : ils sont supprimés de la liste, qui devient ('H', 'G', 'D'). La position courante recule de 1.*
4. *Position courante : 1, les mouvements considérés sont 'G' et 'D' : ils sont supprimés de la liste, qui devient ('H'). La position courante recule de 1.*
5. *Position courante : 0. La fin de la liste est atteinte : terminé.*